Engineering Tripos Part IIA **Module Experiment 3F3**

Random Variables and Random Number Generation

Lab Report

Name:	Joseph Brakoniecki
CRSID:	jdb209
College:	Queens'
Lab session:	18 October 2026

---

1 Uniform random variables

Five numbers from seed 1234

Initialise an LCG random number generator with seed 1234 *and print five uniform random numbers. What numbers do you get?*

```
# Import required libraries
from LCG import LCG
import numpy as np
import matplotlib.pyplot as plt

# Initialize the LCG with a seed value
rng = LCG(1234)

print("Random numbers generated using LCG:")
for i in range(5):
    print(rng.uniform())
```

Output: *0.7143076679203659, 0.20701311994343996, 0.7495418272446841, 0.3460624306462705, 0.8034394562710077*

The same 5 numbers are generated each time the script is run, this makes sense as we start with the same seed and then iteratively compute the LCG equation 5 times. This is a deterministic process.

Histograms

Generate 1,000 *uniform random variables and plot a histogram, then do the same for* 100,000. *Provide detailed comments on what you observe.*

```
# generate uniform numbers
X1 = np.array([rng.uniform() for _ in range(1000)])
X2 = np.array([rng.uniform() for _ in range(100000)])

# Plot histograms
fig, ax = plt.subplots(2)
ax[0].hist(X1, bins=np.linspace(0,1,20))
ax[0].set_title("X1, 1,000 samples")
ax[1].hist(X2, bins=np.linspace(0,1,20))
ax[1].set_title("X2, 100,000 samples")

# Add labels to the axes
ax[0].set_xlabel("x")
ax[0].set_ylabel("Frequency Density")
ax[1].set_xlabel("x")
ax[1].set_ylabel("Frequency Density")
```

```
# Adjust the spacing between subplots
plt.subplots_adjust(hspace=0.5)
plt.show()

# Save the figure as a PDF
fig.savefig("figures/q1-histograms.pdf")
```

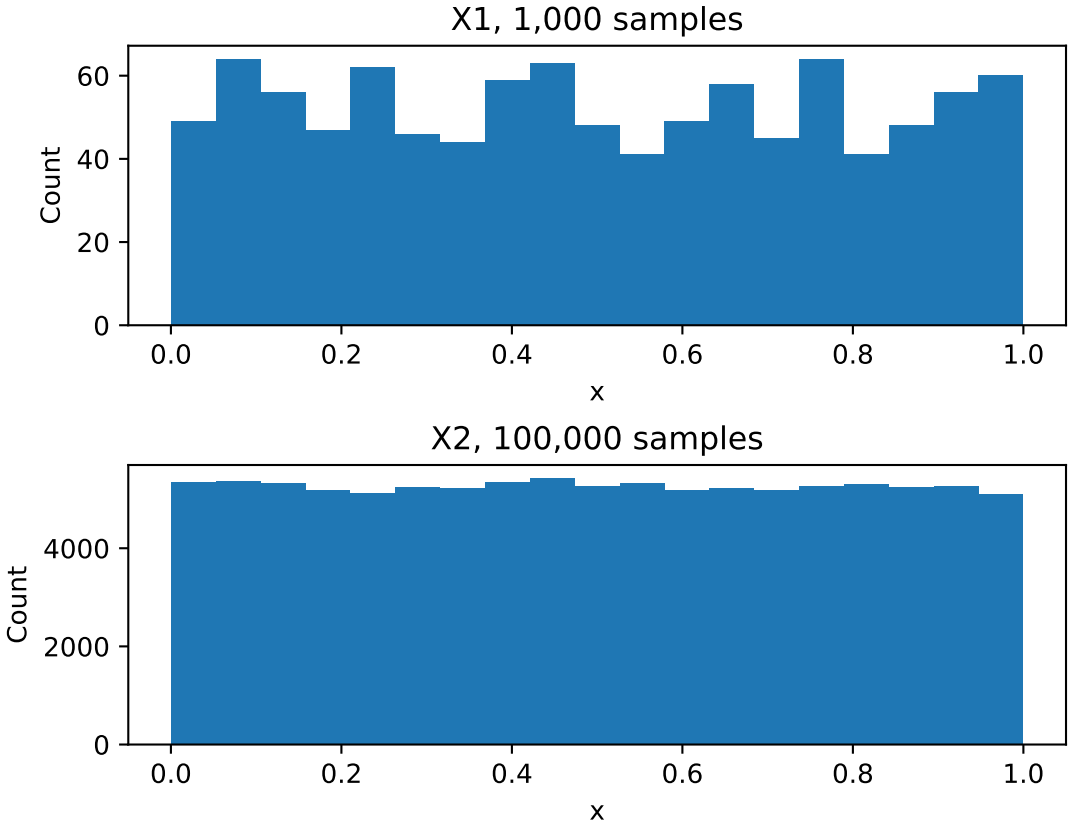


Figure 1: Histograms of 1,000 and 100,000 uniform random numbers generated by the LCG.

We can see that as the number of samples increases the distribution approaches a uniform distribution. This is shown by the histogram approaching a perfect rectangle, with equal numbers of each sample being generated as the number of samples approaches infinity.

Thus by the law of large numbers we can infer that the function is generating random numbers according to a uniform distribution, $U(0,1)$.

Mean, variance and covariance

Calculate the mean and variance of the 100,000 *numbers and compare them with the theoretical values for* $\mathcal{U}(0,1)$. *Compute the covariance between* X_{t+1} *and* X_t. *Why does this provide evidence that the numbers are behaving like uniform random numbers?*

$$X \sim U(0,1)$$
$$f_X(x) = p$$
$$E(X) = \int_{-\infty}^{\infty} x f_X(x)\, dx = \int_0^1 xp\, dx = \left[\frac{px^2}{2}\right]_0^1 = \frac{1}{2}$$
$$E(X^2) = \int_{-\infty}^{\infty} x^2 f_X(x)\, dx = \int_0^1 x^2 p\, dx = \left[\frac{px^3}{3}\right]_0^1 = \frac{1}{3}$$
$$Var(X) = E(X^2) - (E(X))^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12}$$

```
# Calculate mean and variance
mean_X2 = np.mean(X2)
var_X2 = np.var(X2)

print("X1 = 1000 uniform random numbers")
print("X2 = 100000 uniform random numbers")

print(f"Mean of X2: {mean_X2}, Variance of X2: {var_X2}")

# Create X2t and X2tp1 arrays for covariance calculation
X2tp1 = np.copy(X2)
X2t = np.copy(X2)

X2tp1 = X2tp1[1:]
X2t = X2t[:-1]

# Calculate covariance between X2t and X2tp1
# Take the [0,1] (or [1,0]) element of the matrix as this corresponds to the
    covariance between X2t and X2tp1
# Diagonals elements are the variances of X2t and X2tp1, respectively
covarience = np.cov(X2t, X2tp1)[0,1]

print("X2t = Xt values of X2 from index 0 to 99998")
print("X2tp1 = Xt+1 values of X2 from index 1 to 99999")

print(f"Covarience of X2t and X2tp1: {covarience}")
```

	LCG, 100,000 samples	Theory, $\mathcal{U}(0,1)$
Mean	0.49858	0.50000
Variance	0.08327	0.08333

Covariance between X_{t+1} and X_t: 0.00010

We can see that the mean of the LCG and theoretical model is the same to 2 decimal places, with a percentage error of only 0.284%.

The variance of the LCG and theoretical model are the same to 4 decimal places, with a percentage error of only 0.072%.

These are very small errors between the theoretical and real models further implying that LCG follows a uniform distribution. (Law of Large numbers states that as the number of samples increases the sample mean converges to the true mean)

The covariance between X_{t+1} and X_t is essentially 0. This shows that the next number generated by the LCG has no correlation to the previous number generated. And therefore suggesting that the LCG is generating random numbers.

2 Functions of random variables

$y = x^2$

For $x \sim \mathcal{U}(0,1)$ and $y = g(x) = x^2$, calculate $p_Y(y)$ using the Jacobian formula. Verify it by transforming a large collection of samples $x^{(i)}$ to give $y^{(i)} = g(x^{(i)})$, histogramming the y samples and overlaying your formula.

$$X \sim U(0,1)$$

$$f_X(x) = \begin{cases} 1, & 0 \le x \le 1, \\ 0, & \text{otherwise.} \end{cases}$$

$$y = g(x) = x^2$$

$$g^{-1}(y) = \pm\sqrt{y}$$

$$\frac{dy}{dx} = 2x$$

$$f_Y(y) = \sum_{k=1}^{K} \frac{f_X(x)}{\left|\frac{dy}{dx}\right|}\bigg|_{x=x_k(y)} = \frac{f_X(\sqrt{y})}{2\sqrt{y}} + \frac{f_X(-\sqrt{y})}{2\sqrt{y}} = \frac{f_X(\sqrt{y})}{2\sqrt{y}}$$

$$f_Y(y) = \frac{f_X(\sqrt{y})}{2\sqrt{y}} = \begin{cases} \frac{1}{2\sqrt{y}}, & 0 \le y \le 1, \\ 0, & \text{otherwise.} \end{cases}$$

```python
# Import required libraries
from LCG import LCG
import numpy as np
import matplotlib.pyplot as plt

# Initialize the LCG with a seed value
rng = LCG(1234)

# Generate 100,000 uniform random numbers using the LCG
X = np.array([rng.uniform() for _ in range(100000)])

# Calculate Y = X^2
Y = X**2

# Define the PDF of Y
y = np.linspace(0.01,1,1000)
f_Y = 1/(2*y**0.5)

# Plot histogram, set density to True to normalize the histogram
plt.hist(Y, bins=np.linspace(0,1,20), density=True, label = "Normalized Histogram of Y
    ")

# Plot the PDF of Y
plt.plot(y, f_Y, label = "PDF of Y", color = "red")

# Show the plot with labels and legend
plt.legend()
plt.xlabel("y")
plt.ylabel("Frequency Density")

# Save the figure as a PDF
plt.savefig("figures/q2-square.pdf")
```

```
plt.show()
```

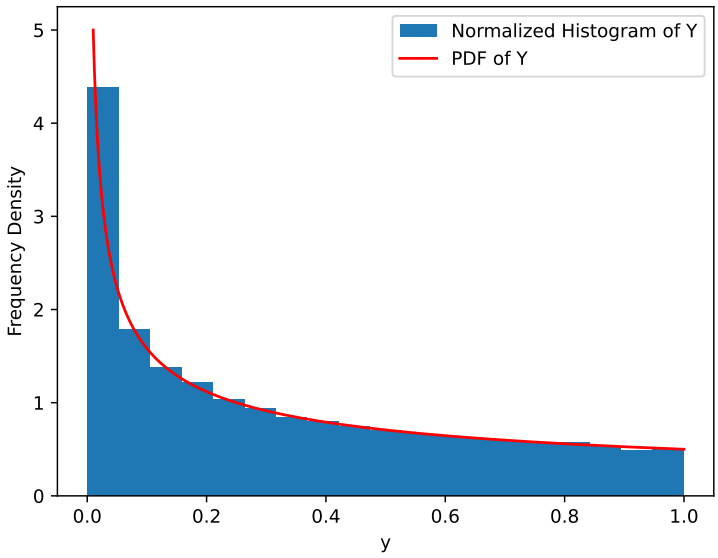


Figure 2: Histogram of $y = x^2$ for transformed uniform samples, with the density from the Jacobian formula overlaid.

We can see that the expected PDF of Y, calculated from the formula, does in fact fit the experimental histogram obtained from squaring 100,000 random numbers from the LCG pseudorandom number generator.

Thus verifying the usage of the Jacobian formula in calculating PDFs.

$y = \ln(x) - \ln(1 - x)$

Calculate $p_Y(y)$ for $y = g(x) = \ln(x) - \ln(1 - x)$ using the Jacobian formula, and again verify your result by histogramming transformed samples.

$$X \sim U(0, 1)$$

$$f_X(x) = \begin{cases} 1, & 0 \leq x \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

$$y = g(x) = \ln(x) - \ln(1 - x)$$

$$\frac{dy}{dx} = \frac{1}{x} + \frac{1}{1 - x} = \frac{1}{x(1 - x)}$$

$$g^{-1}(y) = \frac{e^y}{1 + e^y}$$

$$f_Y(y) = \sum_{k=1}^{K} \left. \frac{f_X(x)}{\left|\frac{dy}{dx}\right|} \right|_{x=x_k(y)} = \frac{f_X\left(\frac{e^y}{1+e^y}\right)}{\frac{1}{\left(\frac{e^y}{1+e^y}\right)\left(1-\frac{e^y}{1+e^y}\right)}} = \left[\frac{e^y}{(1 + e^y)^2}\right] f_X\left(\frac{e^y}{1 + e^y}\right)$$

$$f_Y(y) = \left[\frac{e^y}{(1 + e^y)^2}\right] f_X\left(\frac{e^y}{1 + e^y}\right) = \frac{e^y}{(1 + e^y)^2}$$

```python
# Import required libraries
from LCG import LCG
import numpy as np
import matplotlib.pyplot as plt

# Initialize the LCG with a seed value
rng = LCG(1234)

# Generate 100,000 uniform random numbers using the LCG
X = np.array([rng.uniform() for _ in range(100000)])

# Calculate Y = ln(X) - ln(1-X)
Y = np.log(X) - np.log(1 - X)

# Define the PDF of Y
y = np.linspace(-14,14, 1000)
f_Y = (np.exp(y))/((1+np.exp(y))**2)

# Plot histogram, set density to True to normalize the histogram
plt.hist(Y, bins = 30, density=True, label = "Normalized Histogram of Y")

# Plot the PDF of Y
plt.plot(y, f_Y, label = "PDF of Y", color = "red")

# Show the plot with labels and legend
plt.legend(loc = "upper left", handlelength=1, handletextpad=0.5)
plt.xlabel("y")
plt.ylabel("Frequency Density")

# Save the figure as a PDF
plt.savefig("figures/q2-log.pdf")
plt.show()
```

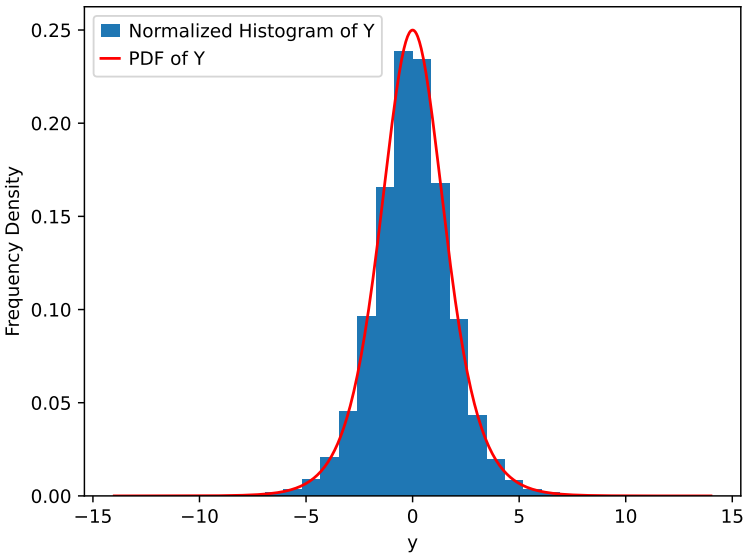


Figure 3: Histogram of $y = \ln(x) - \ln(1-x)$ for transformed uniform samples, with the density from the Jacobian formula overlaid.

We can see that the PDF derived from the Jacobian formula as shown in the mathmatical expression above (noting that this result is unbounded and thus Y can hold the value of any real number) matches the experimental result obtained from applying $Y = \ln(x) - \ln(1-x)$ to 100,000 random numbers generated from the LCG pseudo random number generation.

Further verifying the usage of the Jacobian formula in calculating PDFs.

3 Non-uniform random number generation

Why the inverse CDF method works

To generate random variables with CDF $F(y)$, we generate $x \sim \mathcal{U}(0,1)$ and set $y = F^{-1}(x)$. Why does this work?

This method works because the output of $F(Y)$ is constrained to the range: $0 \leq F(Y) \leq 1$ as it is a probability and so has to be between 0 and 1.

Thus the range of F(Y) lives in the same as the domain of the random variable $X \sim U(0,1)$, i.e. constrained between 0 and 1.

Therefore if the inverse of F, F^{-1}, has the same domain as X. Thus by passing a random input, X as input to F^{-1} we will obtain a random value in the domain of Y.

Thus setting $y = F^{-1}(x)$ allows us to generate random numbers in the distribution defined by Y, with CDF $F(y)$.

CDF and inverse CDF of the exponential distribution

For the exponential distribution with mean one, $p_Y(y) = \exp(-y)$, $y \geq 0$, calculate the CDF and the inverse CDF.

$$
\begin{aligned}
&Y \sim \text{Exp}(1) \\
&f_Y(y) = \begin{cases} e^{-y} & y \geq 0 \\ 0 & \text{otherwise} \end{cases} \\
&F_Y(y) = \int_0^y e^{-y}\, dy = [-e^{-y}]_0^y = 1 - e^{-y} \\
&x = F_Y(y) \implies e^{-y} = 1 - x \\
&\therefore F_Y^{-1}(x) = -\ln(1-x)
\end{aligned}
$$

Sampling from the exponential distribution

Use the inverse CDF method to generate samples from the exponential distribution. Plot histograms or kernel density estimates and overlay them on the exponential density, and comment on the results.

```
# Import required libraries
from LCG import LCG
import numpy as np
import matplotlib.pyplot as plt

# Initialize the LCG with a seed value
rng = LCG(1234)

# Generate 100,000 uniform random numbers using the LCG
X = np.array([rng.uniform() for _ in range(100000)])
```

```
# Using the inverse CDF method to generate random numbers form the distribution of Y ~
    Exp(1)
Y_sample = - np.log(1-X)

# Define the PDF of Y
y = np.linspace(-0, 12, 1000)
Y_pdf = np.exp(-y)

# Plot histogram, set density to True to normalize the histogram
plt.hist(Y_sample, bins = 30, density=True, label = "Y Sampled using the Inverse CDF
    Method")

# Plot the PDF of Y
plt.plot(y, Y_pdf, label = "PDF of Y", color = "red")

# Show the plot with labels and legend
plt.legend(loc = "upper left", handlelength=1, handletextpad=0.5)
plt.xlabel("y")
plt.ylabel("Frequency Density")

# Save the figure as a PDF
plt.savefig("figures/q3-exponential.pdf")
plt.show()
```

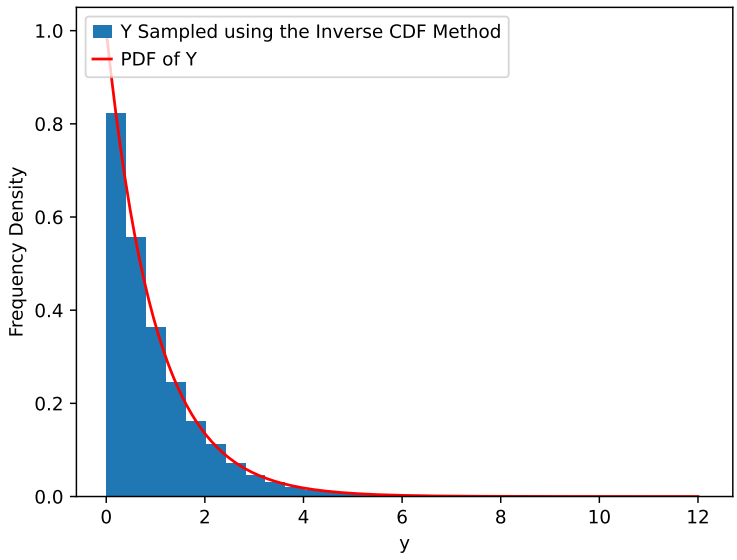


Figure 4: Samples generated by the inverse CDF method, compared with the exponential density $p_Y(y) = \exp(-y)$.

We can clearly see that the histogram is distributed according to the the exponential distribution, as shown by a high degree of agreement between the red line and blue area normalized histogram.

Thus verifying the inverse CDF method for non-uniform random number generation.

4 Simulating a stochastic process

Bernoulli and binomial random variables

Complete the `bernoulli` method of the `LCG` class to generate Bernoulli random variables with parameter p, and add a method that generates binomial random variables.

Bernoulli Distribution
Uniform method generates a random decimal between 0 and 1 with all values holding equal probability. However for a Bernoulli Variable we want it to generate 1 with probability p or 0 with probability $1 - p$.

The probability that a number generated by uniform is less than or equal to p is in fact p. (i.e. $X \sim U(0,1);\ P(X \leq p) = p$) Due to the nature of the uniform distribution generating all values at the same probability.

This means that if the uniform function of the LCG returns a number less than or equal to p, we set bernolli to be a 1. Otherwise we set it to be a 0.

Binomial Distribution A binomial distribution is simply the sum of n Bernolli distributions all with parameter p. Or mathmatically:

$$X \sim \text{Bernoulli}(p)$$
$$Y \sim B(n,p)$$
$$Y = \sum_{r=1}^{n} X_r$$

Where X_r are independent random Bernoulli Variables.

Thus to obtain a random binomial value, we simply have to call the Bernoulli function with probability p, n times and sum up the values. This will then result in a random binomal value distributed as $B(n,p)$.

```
# Linear Congruential Generator (LCG) implementation in Python
class LCG(object):
    def __init__(self, seed):
        # Initialize the LCG with a seed value
        self.__X = int(seed)

    def __call__(self):
        # Update the internal state using the LCG formula
        self.__X = (1664525 * self.__X + 1013904223) % 4294967296
        return self.__X

    def uniform(self):
        # Return a uniform random number in the range [0, 1)
        return self() / 4294967296

    def bernoulli(self, p):
        # Return a Bernoulli random variable with probability p

        # If the uniform random number is less than or equal to p, return 1 otherwise
    return 0
        # This is because the probability of getting a 1 is p,
        # and thus the fraction of decimals between 0 and 1 that are less than or
    equal to p = p
        # Thus resulting in a Bernoulli random variable with probability p
        return 1 if self.uniform() <= p else 0
```

```
    def binomial(self, n, p):
        # Return a Binomial random variable with parameters n and p

        # Initialize a variable to keep track of the total number of successes
        total = 0

        # Binomial is the sum of n independent Bernoulli random variables with
    probability p
        for i in range(n):
            total += self.bernoulli(p)

        # Return the total number of successes in n trials
        return total
```

Neutral selection

Write Python code to simulate $N_A(t+1) \sim \text{Binomial}\big(N, N_A(t)/N\big)$, $N_B(t+1) = N - N_A(t+1)$. *Run simulations with* $N = 200$ *and* $N_A(0) = 100$ *until* $t = 500$, *plot* $N_A(t)$ *and* $N_B(t)$ *over time, and repeat this several times.*

```
# Import required libraries
from LCG import LCG
import numpy as np
import matplotlib.pyplot as plt

# Define a population class to handle the simulation over generations
class Population:
    def __init__(self, N, Na, seed = 1234):

        # Population Size, N
        self.N = N

        # Number of individuals with gene A, Na
        self.Na = Na

        # Number of individuals with gene B, Nb
        self.Nb = N - Na

        # Generation/Time counter
        self.t = 0

        self.rng = LCG(seed)

    def next_generation(self):
        # Calculate the probability of selecting gene A
        p_A = self.Na / self.N

        # Generate the next generation using binomial distribution
        self.Na = self.rng.binomial(self.N, p_A)

        # Update the number of individuals with gene B
        self.Nb = self.N - self.Na

        # Increment the generation counter
        self.t += 1

    def simulate(self, generations, track_history=False):
```

```
        # Simulate the population for a given number of generations

        # If the current gneration counter is greater than or equal to the specified
    number of generations
        # (Which defines simulation length). Return the populaton size of gene A and
    gene B as a dictionary
        # Otherwise generate the next generation and recursivly call the simmulate
    function until the above critera is met
        if self.t >= generations:
            return {"Na": self.Na, "Nb" : self.Nb} if not track_history else self.
    history
        else:

            # If Track history is set to true store how population sizes change over
    time
            # If it is the first generation, initialize the history dictionary
            # Otherwise append current population sizes to the history dictionary
            if track_history == True and self.t == 0:
                self.history = {"t" : [self.t], "Na": [self.Na], "Nb": [self.Nb]}
            elif track_history == True:
                self.history["Na"].append(self.Na)
                self.history["Nb"].append(self.Nb)
                self.history["t"].append(self.t)

            # Generate the next generation and recursively call the simulate function
            self.next_generation()
            return self.simulate(generations, track_history)


# Initialize a population with size N= 200 and Na = 100
population = Population(N = 200, Na = 100, seed = 1234)

# Simulate the population for 500 generations and track the history of population
    sizes
generations = 500
history = population.simulate(generations = generations, track_history = True)

Na = history["Na"]
Nb = history["Nb"]
t = history["t"]

plt.plot(t,Na, label = "Na")
plt.plot(t,Nb, label = "Nb")

# Show the plot with labels and legend
plt.legend(loc = "upper left", handlelength=1, handletextpad=0.5)
plt.xlabel("t")
plt.ylabel("Population Size")

# Save the figure as a PDF
plt.savefig("figures/q4-neutral-selection.pdf")
plt.show()
```

We can clearly see that Gene A is a dominant gene, with it taking over 100% by the end of the simulation. Gene B is recessive with it dying out over large numbers of generations.

Changing the seed value of the LCG changes the speed at which N_a and N_b converge to there final values but the result final result occurs nonetheless.

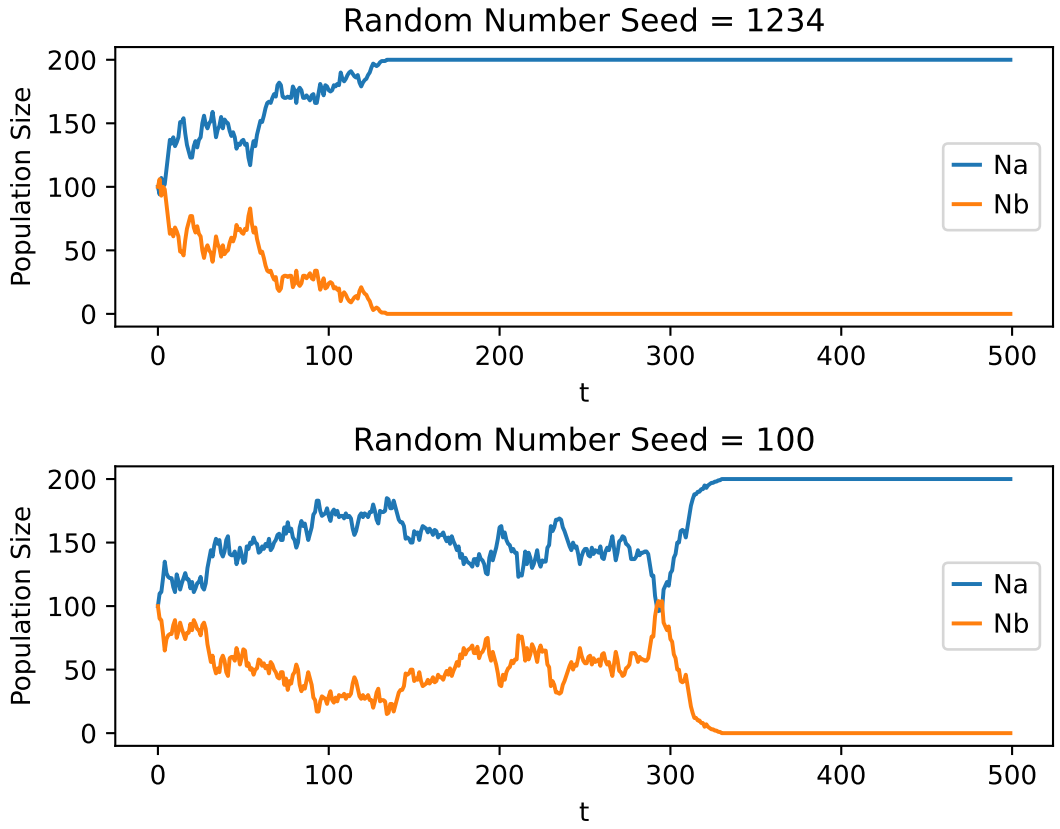


Figure 5: $N_A(t)$ and $N_B(t)$ for several runs of the neutral selection simulation, with $N = 200$ and $N_A(0) = 100$.

This can clearly be observed from the comparison of using seed 1234 and 100, we can see that both converge to the same final values (showing Gene A to be dominant and B to be recessive) but seed 100 converges significantly slower.

This is just because a different sequence of random numbers has been generated resulting in us taking a different path to convergence.